Milestone 2 Report

Smart MCQ Solver — NLP with Hugging Face Transformers

A fully explained walkthrough of the milestone-2.ipynb notebook: every question, the reasoning behind each approach, the exact code used, the notebook's output, and the final answer — written so that anyone unfamiliar with the notebook can understand what was done and why.

Source notebook: milestone-2.ipynb
Dataset: Kaggle "smart-mcq-solver-challenge"

1. Project Overview

This notebook tackles a multiple-choice question answering (MCQ) problem — each row in the dataset provides a question (**prompt**) and five candidate options (**A** through **E**), and the goal is to identify the correct option (**answer**). The notebook is organized as a guided sequence of 13 questions that progressively build up an understanding of the Hugging Face NLP ecosystem, moving from basic dataset handling and tokenization, through BERT's internal architecture and attention mechanism, to modern sentence-embedding retrieval, zero-shot classification, and finally generative answering with a small language model. Together the sections trace the same evolution the field of NLP followed — from lexical methods, to contextual encoders, to instruction-following generative models.

1.1 Environment & Setup

The notebook begins by verifying the software and hardware environment it is running in, which matters a great deal for transformer workloads since model loading and inference speed depend heavily on library versions and GPU availability.

Python	3.10.12
Platform	Linux 6.6.114.1-microsoft-standard-WSL2 (WSL2)
NumPy	2.2.6
Pandas	2.3.3
PyTorch	2.5.1+cu121
Transformers	5.12.0
Device	CUDA (NVIDIA GeForce RTX 3050 Laptop GPU, ~4.29 GB VRAM)

1.2 Dataset

The dataset is read from three CSV files. A quick inspection with **.describe()**, **.shape** and **.isna().sum()** confirms the data is clean (no missing values) before any modeling begins.

Files	train.csv, test.csv, sample_submission.csv
train.csv shape	2000 rows × 8 columns
test.csv shape	500 rows × 7 columns
sample_submission.csv shape	500 rows × 2 columns
Columns	id, prompt, A, B, C, D, E, answer (test.csv omits answer)
Missing values	None — all columns fully populated

2. Table of Contents

Introduction to Hugging Face Transformers & Datasets

- Q1. Building combined_text with the Hugging Face datasets Library
- Q2. Inspecting the BERT Tokenizer's Vocabulary Size
- Q3. Locating the [SEP] Separator Token ID
- Q4. Batch-Tokenizing the Full prompt Column

BERT/RoBERTa Architecture & Attention Mechanisms

- Q5. Computing the Dimensionality of a Single Attention Head
- Q6. Shape of last_hidden_state for a Single Prompt
- Q7. Summing the First 5 Values of the [CLS] Embedding
- Q8. Extracting a Specific Attention Weight (Last Layer, Head 0)

Context-Aware Embeddings

- Q9. Semantic Similarity with Sentence Embeddings (MiniLM)
- Q10. Comparing TF-IDF vs. MiniLM Ranking Pipelines (MAP@3)

Zero-Shot Classification Concepts

- Q11. Zero-Shot Classification of MCQ Options
- Q12. Softmax vs. Independent Sigmoids in Zero-Shot Classification

Generative AI for MCQ Answering

- Q13. Generative Answering with a Small Language Model (FLAN-T5)

3. Introduction to Hugging Face Transformers & Datasets

Question 1: Building combined_text with the Hugging Face datasets Library

*Load train.csv using the Hugging Face **datasets** library (not pandas). Use **.map()** to create a new column **combined_text** that concatenates the prompt and A columns with a space in between. What is the exact character length of combined_text for the row at index 51 (zero-indexed)?*

Approach & Explanation

- The **datasets** library (also from Hugging Face) is the standard way to load and manipulate text corpora for transformer models, and it is used instead of pandas here because it stores data in an Apache Arrow backend that scales to very large datasets and integrates directly with tokenizers and models.
- **load_dataset("csv", data_files=...)** reads the CSV file into a **DatasetDict** containing a single split called *train* by default.
- **.map()** applies a function to every row (or batch of rows) of the dataset and returns a new dataset with the extra column attached — here the function concatenates the **prompt** and **A** fields with a single space between them, producing the new **combined_text** field.
- Finally, indexing the dataset at position 51 and taking Python's built-in **len()** of the string gives the exact number of characters (not tokens) in that row's combined_text.

Code

```
from datasets import load_dataset

# Load train.csv using Hugging face datasets library
dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")

# print the dataset to see its structure
print(dataset)

# Use .map() to create combined_text column by concatenating prompt and A columns
dataset = dataset.map(lambda x: {"combined_text": x["prompt"] + " " + x["A"]})

# Get character length at index 51 (zero-indexed)
print(f"Ans: {len(dataset['train'][51]['combined_text'])}")
```

Notebook Output

```
DatasetDict({
    train: Dataset({
        features: ['id', 'prompt', 'A', 'B', 'C', 'D', 'E', 'answer'],
        num_rows: 2000
    })
})
Ans: 614
```

Character length of combined_text at row 51 = 614

Question 2: Inspecting the BERT Tokenizer's Vocabulary Size

*Initialize the **bert-base-uncased** tokenizer. What is the exact total vocabulary size hardcoded into this tokenizer?*

Approach & Explanation

- A tokenizer converts raw text into a sequence of integer IDs that a transformer model can consume. **BertTokenizer.from_pretrained("bert-base-uncased")** downloads the exact WordPiece vocabulary file that Google used when it originally trained BERT-base.
- The tokenizer's internal **.vocab** dictionary maps every known sub-word token (including the special tokens like [PAD], [CLS], [SEP], [MASK], [UNK] and 994 reserved [unusedN] slots) to a unique integer ID.
- **tokenizer.vocab_size** reports the size of this fixed vocabulary — it is a property of the pretrained checkpoint and does not change no matter what text is tokenized.

Code

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

print(len(tokenizer.vocab))
print(f"Ans: Vocab Size: {tokenizer.vocab_size}")
```

Notebook Output

```
30522
Ans: Vocab Size: 30522
```

bert-base-uncased vocabulary size = 30,522 sub-word tokens

Question 3: Locating the [SEP] Separator Token ID

Transformers rely on special tokens to understand sentence boundaries. Using the bert-base-uncased tokenizer, extract the exact integer ID assigned to the [SEP] (Separator) token.

Approach & Explanation

- BERT was pre-trained on pairs of sentences, so it needs special marker tokens to know where a sequence starts, where one sentence ends and the next begins, and where padding begins.

- [CLS] is prepended to every input and its final hidden state is typically used as an aggregate representation of the whole sequence (useful for classification).
- [SEP] is inserted between two sentences (e.g. a question and an answer, or a CV and a job description) and at the very end of the input, telling the model exactly where a segment boundary lies.
- Each special token has a fixed ID inside the vocabulary; these can be read directly off the tokenizer object via convenience attributes such as **.cls_token_id**, **.sep_token_id**, **.pad_token_id**, **.unk_token_id** and **.mask_token_id**.

Code

```
print(tokenizer.cls_token_id)
print(tokenizer.sep_token_id)
print(tokenizer.pad_token_id)
print(tokenizer.unk_token_id)
print(tokenizer.mask_token_id)

print(f"Separator token ID: {tokenizer.sep_token_id}")
```

Notebook Output

```
101
102
0
100
103
Separator token ID: 102
```

[SEP] token ID = 102 (for reference: [CLS]=101, [PAD]=0, [UNK]=100, [MASK]=103)

Question 4: Batch-Tokenizing the Full prompt Column

*Tokenize the entire **prompt** column of the train dataset simultaneously with **padding='max_length'**, **truncation=True**, **max_length=128** and **return_tensors='pt'**. What is the exact shape of the resulting input_ids tensor?*

Approach & Explanation

- Transformer models require every sequence in a batch to be the same length. **padding="max_length"** pads every tokenized prompt up to a fixed length of 128 tokens with the [PAD] token, while **truncation=True** cuts off any prompt that is longer than 128 tokens.
- Passing the whole column at once (a Python list of 2000 strings) lets the tokenizer batch-process every row in a single call rather than looping row by row, which is both faster and produces one aligned tensor.
- **return_tensors="pt"** tells the tokenizer to return native PyTorch tensors instead of plain Python lists, ready to be fed straight into a PyTorch model.
- Because there are 2000 training rows and each is padded/truncated to exactly 128 tokens, the resulting **input_ids** tensor has shape **(2000, 128)** — one row of 128 token IDs per training example.

Code

```
from datasets import load_dataset
from transformers import BertTokenizer

dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

tokens = tokenizer(
    list(dataset["train"]["prompt"]),
    padding="max_length",
    truncation=True,
    max_length=128,
    return_tensors="pt"
)

print(tokens["input_ids"].shape)
```

Notebook Output

```
torch.Size([2000, 128])
```

input_ids tensor shape = torch.Size([2000, 128]) (2000 rows × 128 tokens each)

4. BERT/RoBERTa Architecture & Attention Mechanisms

Question 5: Computing the Dimensionality of a Single Attention Head

bert-base-uncased has a hidden embedding size of 768 dimensions and uses 12 attention heads per layer. What is the exact dimensionality of each individual attention head?

Approach & Explanation

- In multi-head self-attention, the model does not compute one large attention operation over the full 768-dimensional embedding. Instead the embedding space is split evenly across several smaller "heads" that each attend to different aspects of the relationships between tokens, and their outputs are concatenated back together at the end of the layer.
- The per-head dimensionality is therefore simply the hidden size divided by the number of heads: **768 / 12 = 64**. Splitting the workload this way lets BERT learn several different types of relationships (e.g. syntactic vs. semantic) in parallel while keeping the total number of parameters and compute roughly the same as a single, full-width attention operation.

Code

```
hidden_size = 768
num_heads = 12

head_dim = hidden_size // num_heads
print(head_dim)  # → 64
```

Notebook Output

```
64
```

Each attention head has dimensionality 64 (768 ÷ 12)

Question 6: Shape of last_hidden_state for a Single Prompt

*Load bert-base-uncased with **AutoModel.from_pretrained()**. Tokenize the prompt from row ID 0 with default tokenizer settings (no manual padding/truncation), pass it through the model, and report the exact shape of the last_hidden_state tensor.*

Approach & Explanation

- **AutoModel** loads the bare BERT encoder (no classification head attached), so calling it on tokenized input returns raw hidden states rather than logits.
- Because no padding or truncation is applied, the sequence length is whatever the tokenizer naturally produces for that specific prompt — [CLS], the prompt's sub-word tokens, and a trailing [SEP].

- The model's output object exposes **.last_hidden_state**, a tensor of shape **(batch_size, sequence_length, hidden_size)**. With a batch size of 1 (one prompt), a natural sequence length of 31 tokens for this particular prompt, and BERT-base's hidden size of 768, the resulting shape is **torch.Size([1, 31, 768])** — one 768-dimensional contextual embedding vector for every one of the 31 tokens.

Code

```
from datasets import load_dataset
from transformers import BertTokenizer, AutoModel

dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

# get the prompt from row ID 0
prompt = dataset["train"][0]["prompt"]

# Tokenize with default settings (no padding/truncation)
tokens = tokenizer(prompt, return_tensors="pt")

# Pass through model to get hidden states
output = model(**tokens)

print(f"Ans: Shape of the last hidden state: {output.last_hidden_state.shape}")
```

Notebook Output

```
Ans: Shape of the last hidden state: torch.Size([1, 31, 768])
```

last_hidden_state shape = torch.Size([1, 31, 768]) (batch=1, tokens=31, hidden=768)

Question 7: Summing the First 5 Values of the [CLS] Embedding

Using the last_hidden_state tensor from Q6, extract the embedding vector for the [CLS] token (index 0). What is the sum of the first 5 float values in this vector, rounded to 4 decimal places?

Approach & Explanation

- The [CLS] token always sits at position 0 of the sequence dimension, so **last_hidden_state[0, 0, :]** slices out (batch index 0, token index 0, all 768 hidden dimensions) — the full contextual embedding vector that BERT computed for the [CLS] token.
- This 768-dimensional vector is often used downstream as a compact summary of the entire input sequence for classification tasks (that is exactly why [CLS] exists).
- Slicing the first 5 elements of that vector and summing them with **.sum().item()** converts the small PyTorch tensor sum into a plain Python float, which is then rounded to 4 decimal places for reporting.

Code

```
# get [CLS] token = index 0 of sequence dimension
cls_embedding = output.last_hidden_state[0, 0, :]  # batch, token_index, hidden
# Sum of first 5 values of the CLS embedding
```

```
first_5 = cls_embedding[:5]
print(first_5)
print(f"Ans: Sum of first 5 values of the CLS embedding: {round(first_5.sum().item(), 4)}")
```

Notebook Output

```
tensor([-0.4677, -0.0754, -0.2019, -0.0071, -0.4480], grad_fn=<SliceBackward0>)
Ans: Sum of first 5 values of the CLS embedding: -1.2001
```

Sum of first 5 values of the [CLS] embedding = -1.2001

Question 8: Extracting a Specific Attention Weight (Last Layer, Head 0)

*Load bert-base-uncased with **output_attentions=True**. Tokenize "Light-ion fusion is a technique." and pass it through the model. From the last layer (index -1), first head (index 0), what attention weight does the [CLS] token pay to the word "fusion"?*

Approach & Explanation

- Setting **output_attentions=True** makes the model additionally return every layer's raw attention-weight matrices inside **output.attentions**, a tuple with one tensor per transformer layer (12 for BERT-base). Each tensor has shape (batch, num_heads, seq_len, seq_len), where entry [i, j] is how much token i "attends to" token j.
- The code first converts the input_ids back into readable sub-word tokens with **convert_ids_to_tokens** to visually confirm the tokenization: ['[CLS]', 'light', '-', 'ion', 'fusion', 'is', 'a', 'technique', '.', '[SEP]']. This shows that "fusion" happens to occupy position 4 in the sequence.
- **output.attentions[-1]** selects the last (12th) transformer layer, **[0]** selects the first item in the batch, and **[0]** again selects attention head 0 — leaving a 10×10 attention matrix for this 10-token sentence.
- Reading **attention[0, fusion_idx]** gives the attention weight that the [CLS] token (row 0, since it is the query) assigns to the "fusion" token (column 4, the key) — i.e. how strongly the summary token focuses on that specific word.

Code

```
from transformers import BertTokenizer, AutoModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased", output_attentions=True)

# tokenize the exact string
text = "Light-ion fusion is a technique."
tokens = tokenizer(text, return_tensors="pt")

# Check token ids and find index of "fusion"
print(tokenizer.convert_ids_to_tokens(tokens["input_ids"][0]))

# Pass through model
output = model(**tokens)

# Extract last layer, first head attention
attention = output.attentions[-1][0][0]  # last layer, first head
```

```
# Find fusion token index from printed tokens above
fusion_idx = tokenizer.convert_ids_to_tokens(tokens["input_ids"][0]).index("fusion")
print(f"Fusion token index: {fusion_idx}")

cls_to_fusion = attention[0, fusion_idx]
print(f"Ans: Attention from CLS to Fusion: {round(cls_to_fusion.item(), 4)}")
```

Notebook Output

```
['[CLS]', 'light', '-', 'ion', 'fusion', 'is', 'a', 'technique', '.', '[SEP]']
Fusion token index: 4
Ans: Attention from CLS to Fusion: 0.1025
```

Attention weight [CLS] → "fusion" (last layer, head 0) = 0.1025

5. Context-Aware Embeddings

Question 9: Semantic Similarity with Sentence Embeddings (MiniLM)

Initialize sentence-transformers/all-MiniLM-L6-v2. Encode the prompt and Option B for row ID 0, then compute the cosine similarity between the two vectors with util.cos_sim(). What is the resulting similarity score, rounded to 4 decimal places?

Approach & Explanation

- Unlike raw BERT, sentence-transformer models such as MiniLM-L6-v2 are specifically fine-tuned so that **.encode()** produces a single fixed-size vector per sentence whose geometry is meaningful for comparison — semantically similar sentences end up close together in the embedding space, unlike raw token-level BERT embeddings.
- The prompt (a question) and Option B (a candidate answer) for row 0 are each encoded independently into dense vectors.
- **util.cos_sim()** computes the cosine of the angle between the two vectors — a value between -1 and 1 where 1 means the vectors point in exactly the same direction (maximally similar) and 0 means they are unrelated.
- The measured similarity of 0.7658 indicates the prompt and Option B share strong semantic overlap, which is exactly the signal later reused to rank multiple-choice options by relevance in Q10.

Code

```
from datasets import load_dataset
from sentence_transformers import SentenceTransformer, util

dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")

# Get asked data from row ID 0
row_id0 = dataset["train"][0]
prompt = row_id0["prompt"]
option_b = row_id0["B"]

# Load model
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# Get embeddings for prompt and option B
prompt_emd = model.encode(prompt)
option_b_emd = model.encode(option_b)

# Compute cosine similarity
score = util.cos_sim(prompt_emd, option_b_emd)
print(round(score.item(), 4))
```

Notebook Output

```
Prompt: Pick the best possible answer: What is Martin Heidegger's view on the
relationship between time and human existence? among the listed options.
Option B: Martin Heidegger believes that humans do not exist inside time, but
that they are time. The relationship to the past is a present awareness...
0.7658
```

Cosine similarity (prompt, Option B) for row 0 = 0.7658

Question 10: Comparing TF-IDF vs. MiniLM Ranking Pipelines (MAP@3)

Build two full ranking pipelines over the entire training set: (1) TF-IDF cosine similarity from Milestone 1, and (2) MiniLM sentence embeddings ranked by cosine similarity, producing Top-3 predictions for each. What is the final MAP@3 of the MiniLM pipeline, and how many questions does MiniLM answer correctly in its Top-3 where TF-IDF does not?

Approach & Explanation

- **MAP@3** (Mean Average Precision at 3) rewards a pipeline both for getting the correct option into its Top-3 predictions and for ranking it as high as possible within those three guesses. The helper **ap_at_3()** returns 1/(rank+1) if the true answer is inside the Top-3 list, otherwise 0, and the final MAP@3 is simply the average of that score across all 2000 rows.
- **Pipeline 1 (TF-IDF)** fits a TF-IDF vectorizer on all prompt+options text, then for every row computes the cosine similarity between the TF-IDF vector of the prompt and each of the five options, keeping the top three highest-scoring options as the prediction. TF-IDF is a purely lexical, word-overlap based method — it has no notion of meaning beyond shared vocabulary.
- **Pipeline 2 (MiniLM)** instead encodes the prompt and every option into dense MiniLM sentence embeddings and ranks options by cosine similarity in that semantic space, so it can recognize a correct answer even when it uses completely different wording than the prompt.
- Comparing the two pipelines' MAP@3 scores directly measures how much benefit semantic (embedding-based) retrieval provides over simple keyword overlap. Counting the rows where MiniLM's Top-3 contains the correct answer but TF-IDF's does not further isolates specifically the cases MiniLM "rescues" thanks to its deeper semantic understanding.
- The results show MiniLM (MAP@3 = 0.4231) substantially outperforms TF-IDF (MAP@3 = 0.2962), and it correctly rescues 502 questions that pure keyword matching missed — strong evidence that semantic embeddings are far better suited to this MCQ-answering task than lexical similarity alone.

Code

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer, util

train = pd.read_csv("../dataset/smart-mcq-solver-challenge/train.csv")
options = ["A", "B", "C", "D", "E"]

def ap_at_3(actual, predicted):
    if actual in predicted[:3]:
        return 1 / (predicted[:3].index(actual) + 1)
    return 0

# Pipeline 1: TF-IDF + Cosine Similarity
combine_text = (
```

```
    train["prompt"].fillna("") + " " + train["A"].fillna("") + " " +
    train["B"].fillna("") + " " + train["C"].fillna("") + " " +
    train["D"].fillna("") + " " + train["E"].fillna("")
)

vectorizer = TfidfVectorizer(stop_words="english")
vectorizer.fit(combine_text)

tfidf_top3_list, tfidf_scores = [], []
for _, row in train.iterrows():
    prompt_vec = vectorizer.transform([str(row["prompt"])])
    sims = []
    for opt in options:
        sim = cosine_similarity(prompt_vec, vectorizer.transform([str(row[opt])]))[0][0]
        sims.append((opt, sim))
    sims.sort(key=lambda x: x[1], reverse=True)
    top3 = [x[0] for x in sims[:3]]
    tfidf_top3_list.append(top3)
    tfidf_scores.append(ap_at_3(row["answer"], top3))

tfidf_map3 = sum(tfidf_scores) / len(tfidf_scores)
print(f"TF-IDF MAP@3 = {round(tfidf_map3, 4)}")

# Pipeline 2: MiniLM + Cosine Similarity
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

miniLM_top3_list, miniLM_scores = [], []
for _, row in train.iterrows():
    prompt_emb = model.encode(str(row["prompt"]))
    sims = []
    for opt in options:
        opt_emb = model.encode(str(row[opt]))
        sim = util.cos_sim(prompt_emb, opt_emb).item()
        sims.append((opt, sim))
    sims.sort(key=lambda x: x[1], reverse=True)
    top3 = [x[0] for x in sims[:3]]
    miniLM_top3_list.append(top3)
    miniLM_scores.append(ap_at_3(row["answer"], top3))

miniLM_map3 = sum(miniLM_scores) / len(miniLM_scores)
print(f"MiniLM MAP@3 = {round(miniLM_map3, 4)}")

# Count rows where MiniLM succeeds but TF-IDF fails
rescued = sum(
    1 for i, row in train.reset_index(drop=True).iterrows()
    if row["answer"] not in tfidf_top3_list[i] and row["answer"] in miniLM_top3_list[i]
)
print(f"MiniLM wins over TF-IDF count = {rescued}")
```

Notebook Output

```
TF-IDF MAP@3 = 0.2962
MiniLM MAP@3 = 0.4231
MiniLM wins over TF-IDF count = 502
```

MiniLM MAP@3 = 0.4231 | Questions rescued by MiniLM but missed by TF-IDF = 502

6. Zero-Shot Classification Concepts

Question 11: Zero-Shot Classification of MCQ Options

Initialize the Hugging Face zero-shot-classification pipeline (defaults to facebook/bart-large-mnli). For row index 1, pass Options A, B and C as candidate_labels. What probability score does the top-ranked option receive, rounded to 4 decimal places?

Approach & Explanation

- Zero-shot classification lets a model assign a piece of text to one of several candidate labels without ever being fine-tuned on those specific labels. Under the hood, **facebook/bart-large-mnli** reframes the task as natural-language inference: for each candidate label it asks "does this premise (the prompt) entail this hypothesis (the option)?" and turns the entailment score into a class probability.
- Here the three candidate_labels are simply the text of Options A, B and C for row 1, so the model is effectively being asked to judge which of the three answer options is best entailed by (best supported by) the question.
- By default the pipeline applies a **softmax** across all candidate labels, so the returned scores sum to 1 and can be read as a probability distribution over "which single option is correct."
- The pipeline returns labels sorted from most to least likely; **result["labels"][0]** and **result["scores"][0]** give the top-ranked option and its associated probability.

Code

```
from transformers import pipeline
from datasets import load_dataset

# Load dataset
dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")
row = dataset["train"][1]  # index 1 = 2nd row

# Initialize zero-shot classification pipeline
classifier = pipeline("zero-shot-classification")

# Get prompt and options A, B and C
prompt = row["prompt"]
candidate_labels = [row["A"], row["B"], row["C"]]

# Run classification
result = classifier(prompt, candidate_labels)

print("Top label:", result["labels"][0])
print("Top score:", round(result["scores"][0], 4))
```

Notebook Output

```
Top label: Accelerator-based light-ion fusion is a technique that uses particle
accelerators to achieve particle kinetic energies sufficient to induce
light-ion fusion reactions...
Top score: <value printed in notebook run>
```

Top-ranked option's softmax probability for row 1 (exact value depends on the bart-large-mnli run; see the printed "Top score" in the notebook output)

Question 12: Softmax vs. Independent Sigmoids in Zero-Shot Classification

*Repeat the same zero-shot classification with **multi_label=True**. What is the absolute difference between the sum of the 3 probabilities from Q11 (Softmax) and the sum of the 3 probabilities here (independent Sigmoids)?*

Approach & Explanation

- When **multi_label=False** (the Q11 default), the pipeline treats the candidate labels as mutually exclusive and applies a single softmax across all of them, so the three scores are forced to add up to exactly 1.0 — this is appropriate when exactly one label must be true.
- Setting **multi_label=True** instead treats every candidate label as an independent binary yes/no question and applies a separate sigmoid to each one. Because the labels are no longer forced to compete against each other for a shared probability budget, the three sigmoid scores can independently be low, high, or anywhere in between — their sum is not constrained to equal 1.
- In this example the model was not confident that *any* of the three options was independently entailed by the prompt, so all three sigmoid scores came out close to zero, summing to just 0.0005 versus the softmax sum of 1.0 — an absolute difference of 0.9995. This nicely illustrates how differently the same underlying model scores behave depending on whether the labels are assumed to be mutually exclusive (softmax) or independent (sigmoid).

Code

```
# Run with multi_label=True (Sigmoid instead of Softmax)
result_multi = classifier(prompt, candidate_labels, multi_label=True)

print("Scores:", [round(s, 4) for s in result_multi["scores"]])
print("Sum:", round(sum(result_multi["scores"]), 4))

# Previous question sum (Softmax)
prev_scores_sum = round(sum(result["scores"]), 4)
print("Previous sum (Softmax):", prev_scores_sum)

# Absolute difference
diff = abs(prev_scores_sum - sum(result_multi["scores"]))
print("Absolute difference:", round(diff, 4))
```

Notebook Output

```
Scores: [0.0005, 0.0, 0.0]
Sum: 0.0005
Previous sum (Softmax): 1.0
Absolute difference: 0.9995
```

|Softmax sum - Sigmoid sum| = |1.0000 - 0.0005| = 0.9995

---

7. Generative AI for MCQ Answering

Question 13: Generative Answering with a Small Language Model (FLAN-T5)

Load google/flan-t5-small via the text2text-generation pipeline. For row index 0, construct the exact prompt string "Question: [prompt]. Is the correct answer A: [A] or B: [B]? Answer with just the letter A or B." and generate with max_new_tokens=5. What is the exact string returned by the model?

Approach & Explanation

- This question moves from purely discriminative approaches (ranking/classifying existing options) to a **generative** approach, where the language model is asked to directly produce the answer text itself.
- **google/flan-t5-small** is an instruction-tuned sequence-to-sequence model — it has been trained specifically to follow natural-language instructions, which makes it well suited to a prompt phrased as a direct question with explicit answer-format instructions ("Answer with just the letter A or B").
- The prompt string is built manually by interpolating the row's prompt text and the text of options A and B directly into a natural-language instruction, essentially turning the multiple-choice problem into a short-answer generation task for the model.
- **max_new_tokens=5** caps the length of the generated answer, which is appropriate here since a single letter answer needs only a handful of tokens; capping generation length also keeps inference fast.
- In the executed run this cell raised a **ModuleNotFoundError** for **T5Tokenizer** because the **sentencepiece** package (a dependency of T5-family tokenizers) had not yet been installed in the environment — visible from the very next cell in the notebook, which installs **sentencepiece** to fix this exact issue for a subsequent run.

Code

```
from transformers import pipeline
from datasets import load_dataset

dataset = load_dataset("csv", data_files="../dataset/smart-mcq-solver-challenge/train.csv")
row = dataset["train"][0]  # index 0

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "google/flan-t5-small"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

input_text = (
    f"Question: {row['prompt']}. "
    f"Is the correct answer A: {row['A']} or B: {row['B']}? "
    f"Answer with just the letter A or B."
)

inputs = tokenizer(input_text, return_tensors="pt")
```

```
outputs = model.generate(**inputs, max_new_tokens=5)

result = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(result)
```

Notebook Output

```
ModuleNotFoundError: Could not import module 'T5Tokenizer'. Are this
object's requirements defined correctly?

# Fixed in the very next cell by installing the missing dependency:
!pip install sentencepiece
```

Cell raised ModuleNotFoundError (missing sentencepiece dependency for the T5 tokenizer); the notebook installs sentencepiece immediately afterward to resolve it.

8. Summary of Answers

The table below collects every question's final answer in one place for quick reference.

Q#	Answer
Q1	Character length of combined_text at row 51 = 614
Q2	bert-base-uncased vocabulary size = 30,522 sub-word tokens
Q3	[SEP] token ID = 102 (for reference: [CLS]=101, [PAD]=0, [UNK]=100, [MASK]=103)
Q4	input_ids tensor shape = torch.Size([2000, 128]) (2000 rows × 128 tokens each)
Q5	Each attention head has dimensionality 64 (768 ÷ 12)
Q6	last_hidden_state shape = torch.Size([1, 31, 768]) (batch=1, tokens=31, hidden=768)
Q7	Sum of first 5 values of the [CLS] embedding = -1.2001
Q8	Attention weight [CLS] → "fusion" (last layer, head 0) = 0.1025
Q9	Cosine similarity (prompt, Option B) for row 0 = 0.7658
Q10	MiniLM MAP@3 = 0.4231 | Questions rescued by MiniLM but missed by TF-IDF = 502
Q11	Top-ranked option's softmax probability for row 1 (exact value depends on the bart-large-mnli run; see the printed "Top score" in the notebook output)
Q12	|Softmax sum - Sigmoid sum| = |1.0000 - 0.0005| = 0.9995
Q13	Cell raised ModuleNotFoundError (missing sentencepiece dependency for the T5 tokenizer); the notebook installs sentencepiece immediately afterward to resolve it.

Key Takeaways

- BERT-base tokenizes text into 30,522 possible sub-word units and organizes every input around special tokens ([CLS], [SEP], [PAD], [UNK], [MASK]) that mark sequence and segment boundaries.
- Multi-head self-attention splits the 768-dimensional hidden representation into 12 heads of 64 dimensions each, letting the model learn several types of token relationships in parallel; attention weights can be inspected directly to see which words a token "looks at."
- Sentence-embedding models like MiniLM produce a single semantically meaningful vector per sentence and dramatically outperform lexical TF-IDF similarity for this MCQ task (MAP@3 of 0.4231 vs. 0.2962), because they can match meaning even when wording differs.

- Zero-shot classification (BART-MNLI) can rank candidate answers without any task-specific fine-tuning, and the choice between softmax and independent sigmoids changes whether scores are forced to sum to 1 (mutually exclusive labels) or are free to vary independently (multi-label judgments).
- Instruction-tuned generative models like FLAN-T5 offer a third paradigm — directly generating an answer letter from a natural-language prompt — though they depend on extra tokenizer dependencies (e.g. sentencepiece) being installed correctly.